Tarea 10

Eduardo Verdezoto

In [323…
```python
# ----------------------------- logging --------------------------
import logging
from sys import stdout
from datetime import datetime

logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s][%(levelname)s] %(message)s",
    stream=stdout,
    datefmt="%m-%d %H:%M:%S",
)
logging.info(datetime.now())
```
```
[01-25 12:44:50][INFO] 2026-01-25 12:44:50.106418
```

1)

In [324…
```python
import numpy as np
```

Realice las siguientes multiplicaciones matriz-matriz:

a.

$$\begin{bmatrix}2 & -3\\3 & -1\end{bmatrix}\begin{bmatrix}1 & 5\\2 & 0\end{bmatrix}$$

In [325…
```python
A = np.array([[2, -3],
              [3, -1]])

B = np.array([[1, 5],
              [2, 0]])

C = A @ B
print(C)
```
```
[[-4 10]
 [ 1 15]]
```

b.

$$\begin{bmatrix}2 & -3\\3 & -1\end{bmatrix}\begin{bmatrix}1 & 5 & -4\\-3 & 2 & 0\end{bmatrix}$$

In [326…
```python
A = np.array([[2, -3],
              [3, -1]])

B = np.array([[1, 5, -4],
              [-3, 2, 0]])

C = A @ B
print(C)
```
```
[[ 11   4  -8]
 [  6  13 -12]]
```

c.

$$\begin{bmatrix}2 & -3 & 1\\4 & 3 & 0\\5 & 2 & -4\end{bmatrix}\begin{bmatrix}0 & 1 & -2\\1 & 0 & -1\\2 & 3 & -2\end{bmatrix}$$

In [327…
```python
A = np.array([[2, -3, 1],
              [4, 3, 0],
              [5, 2, -4]])

B = np.array([[0, 1, -2],
              [1, 0, -1],
              [2, 3, -2]])

C = A @ B
print(C)
```
```
[[ -1   5  -3]
 [  3   4 -11]
 [ -6  -7  -4]]
```

d.

$$\begin{bmatrix}2 & 1 & 2\\-2 & 3 & 0\\2 & -1 & 3\end{bmatrix}\begin{bmatrix}1 & 2\\-4 & 1\\0 & 2\end{bmatrix}$$

In [328…
```python
A = np.array([[2, 1, 2],
              [-2, 3, 0],
              [2, -1, 3]])

B = np.array([[1, -2],
              [-4, 1],
              [0, 2]])

C = A @ B
print(C)
```
```
[[ -2   1]
 [-14   7]
 [  6   1]]
```

2)

Determine cuáles de las siguientes matrices son no singulares y calcule la inversa de esas matrices:

In [329…
```python
# ####################################################################
def gauss_jordan(A: np.ndarray) -> np.ndarray:
    """Realiza la eliminación de Gauss-Jordan

    ## Parameters

    ``A``: matriz del sistema de ecuaciones lineales. Debe ser de tamaño n-by-(k), donde n es el número de incógnitas.

    ## Return

    ``A``: matriz reducida por filas.

    """
    if not isinstance(A, np.ndarray):
        logging.debug("Convirtiendo A a numpy array.")
        A = np.array(A, dtype=float)
    n = A.shape[0]

    for i in range(0, n):  # loop por columna

        # --- encontrar pivote
        p = None  # default, first element
        for pi in range(i, n):
            if A[pi, i] == 0:
                # must be nonzero
                continue

            if p is None:
                # first nonzero element
                p = pi
                continue

            if abs(A[pi, i]) < abs(A[p, i]):
                p = pi

        if p is None:
            # no pivot found.
            raise ValueError("No existe solución única.")

        if p != i:
            # swap rows
            logging.debug(f"Intercambiando filas {i} y {p}")
            _aux = A[i, :].copy()
            A[i, :] = A[p, :].copy()
            A[p, :] = _aux

        # --- Eliminación: loop por fila
        # for j in range(i + 1, n): # Eliminación gaussiana
        for j in range(n):  # Gauss-Jordan
            if j == i:
                continue  # skip pivot row

            m = A[j, i] / A[i, i]
            A[j, i:] = A[j, i:] - m * A[i, i:]

            # dividir para la diagonal
        A[i, :] = A[i, :] / A[i, i]

        logging.info(f"\n{A}")

    if A[n - 1, n - 1] == 0:
        raise ValueError("No existe solución única.")



    return A
```

In [330…
```python
def matriz_inversa(A: np.ndarray) -> np.ndarray:
    """Calcula la matriz inversa de A usando eliminación de Gauss-Jordan

    ## Parameters

    ``A``: matriz cuadrada a invertir.

    ## Return

    ``A_inv``: matriz inversa de A.

    """
    if not isinstance(A, np.ndarray):
        logging.debug("Convirtiendo A a numpy array.")
        A = np.array(A, dtype=float)

    n = A.shape[0]

    A_aumentada = np.hstack((A, np.eye(n)))

    A_reducida = gauss_jordan(A_aumentada)

    A_inversa = A_reducida[:, n:]

    return A_inversa
```

In [331…
```python
import numpy as np

def es_no_singular_numpy(A):
    return np.linalg.det(A) != 0
```

a.

$$\begin{bmatrix}4 & 2 & 6\\3 & 0 & 7\\-2 & -1 & -3\end{bmatrix}$$

In [332…
```python
A = np.array([[4, 2, 6],
              [3, 0, 7],
              [-2, -1, -3]])
```

In [333…
```python
es_no_singular_numpy(A)
```

Out[333…
```
np.False_
```

b.

$$\begin{bmatrix}1 & 2 & 0\\2 & 1 & -1\\3 & 1 & 1\end{bmatrix}$$

In [334…
```python
B = np.array([[1, 2, 0],
              [2, 1, -1],
              [3, 1, 1]])
```

In [335…
```python
es_no_singular_numpy(B)
```

Out[335…
```
np.True_
```

In [336…
```python
matriz_inversa(B)
```
```
[01-25 12:44:50][INFO]
[[ 1.  2.  0.  1.  0.  0.]
 [ 0. -3. -1. -2.  1.  0.]
 [ 0. -5.  1. -3.  0.  1.]]
[01-25 12:44:50][INFO]
[[ 1.          0.         -0.66666667 -0.33333333  0.66666667  0.        ]
 [-0.          1.          0.33333333  0.66666667 -0.33333333 -0.        ]
 [ 0.          0.          2.66666667  0.33333333 -1.66666667  1.        ]]
[01-25 12:44:50][INFO]
[[ 1.     0.     0.    -0.25   0.25   0.25 ]
 [-0.     1.     0.     0.625 -0.125 -0.125]
 [ 0.     0.     1.     0.125 -0.625  0.375]]
```

Out[336…
```
array([[-0.25 ,  0.25 ,  0.25 ],
       [ 0.625, -0.125, -0.125],
       [ 0.125, -0.625,  0.375]])
```

c.

$$\begin{bmatrix}1 & 1 & -1 & 1\\1 & 2 & -4 & -2\\2 & 1 & 1 & 5\\-1 & 0 & -2 & -4\end{bmatrix}$$

In [337…
```python
C= np.array([[1, 1, -1, 1],
             [1, 2, -4, -2],
             [2, 1, 1, 5],
             [-1, 0, -2, -4]])
```

In [338…
```python
es_no_singular_numpy(C)
```

Out[338…
```
np.False_
```

d.

$$\begin{bmatrix}4 & 0 & 0 & 0\\6 & 7 & 0 & 0\\9 & 11 & 1 & 0\\5 & 4 & 1 & 1\end{bmatrix}$$

In [339…
```python
D = np.array([[4, 0, 0, 0],
             [6, 7, 0, 0],
             [9, 11, 1, 0],
             [5, 4, 1, 1]])
```

In [340…
```python
es_no_singular_numpy(D)
```

Out[340…
```
np.True_
```

In [341…
```python
matriz_inversa(D)
```
```
[01-25 12:44:50][INFO]
[[ 1.    0.    0.    0.    0.25  0.    0.    0.  ]
 [ 0.    7.    0.    0.   -1.5   1.    0.    0.  ]
 [ 0.   11.    1.    0.   -2.25  0.    1.    0.  ]
 [ 0.    4.    1.    1.   -1.25  0.    0.    1.  ]]
[01-25 12:44:50][INFO]
[[ 1.      0.      0.      0.      0.25    0.      0.      0.    ]
 [ 0.      1.      0.25    0.25   -0.3125  0.      0.      0.25  ]
 [ 0.      0.     -1.75   -2.75    1.1875  0.      1.     -2.75  ]
 [ 0.      0.     -1.75   -1.75    0.6875  1.      0.     -1.75  ]]
[01-25 12:44:50][INFO]
[[ 1.          0.          0.          0.          0.25        0.
   0.          0.        ]
 [ 0.          1.          0.         -0.14285714 -0.14285714  0.
   0.14285714 -0.14285714]
 [-0.         -0.          1.          1.57142857 -0.67857143 -0.
  -0.57142857  1.57142857]
 [ 0.          0.          0.          1.         -0.5         1.
  -1.          1.        ]]
[01-25 12:44:50][INFO]
[[ 1.          0.          0.          0.          0.25        0.
   0.          0.        ]
 [ 0.          1.          0.          0.         -0.21428571  0.14285714
   0.          0.        ]
 [-0.         -0.          1.          0.          0.10714286 -1.57142857
   1.          0.        ]
 [ 0.          0.          0.          1.         -0.5         1.
  -1.          1.        ]]
```

Out[341…
```
array([[ 0.25      ,  0.        ,  0.        ,  0.        ],
       [-0.21428571,  0.14285714,  0.        ,  0.        ],
       [ 0.10714286, -1.57142857,  1.        ,  0.        ],
       [-0.5       ,  1.        , -1.        ,  1.        ]])
```

3)

Resuelva los sistemas lineales 4 x 4 que tienen la misma matriz de coeficientes:

In [342…
```python
from src import descomposicion_LU, resolver_LU
```

In [343…
```python
A = np.array([[1, -1, 2, -1],
             [1, 0, -1, 1],
             [2, 1, 3, -4],
             [0, -1, 1, -1]])
```

$x_1 - x_2 + 2x_3 - x_4 = 6$
$x_1 - x_3 + x_4 = 4$
$2x_1 + x_2 + 3x_3 - 4x_4 = -2$
$-x_2 + x_3 - x_4 = 5$

$$b1 = \begin{bmatrix}6\\4\\-2\\5\end{bmatrix}$$

In [344…
```python
L1, U1 = descomposicion_LU(np.array(A))
b1 = np.array([6,4,-2,5])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  3. -1. -2.]
 [ 0. -1.  1. -1.]]
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0. -2.  1.]]
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0.  0. -1.]]
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0.  0. -1.]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[ 6.]
 [-2.]
 [-8.]
 [ 1.]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 2
[01-25 12:44:50][INFO] suma = [8.]
[01-25 12:44:50][INFO] U[i, i] = 8.0
[01-25 12:44:50][INFO] y[i] = [-8.]
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [4.]
[01-25 12:44:50][INFO] U[i, i] = 1.0
[01-25 12:44:50][INFO] y[i] = [-2.]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [3.]
[01-25 12:44:50][INFO] U[i, i] = 1.0
[01-25 12:44:50][INFO] y[i] = [6.]
Matriz L:
[[ 1.    0.    0.    0.  ]
 [ 1.    1.    0.    0.  ]
 [ 2.    3.    1.    0.  ]
 [ 0.   -1.   -0.25  1.  ]]
Matriz U:
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0.  0. -1.]]
Solución del sistema Ax=b:
[[ 3.]
 [-6.]
 [-2.]
 [-1.]]
```

$x_1 - x_2 + 2x_3 - x_4 = 1$
$x_1 - x_3 + x_4 = 1$
$2x_1 + x_2 + 3x_3 - 4x_4 = 2$
$-x_2 + x_3 - x_4 = -1$

$$b1 = \begin{bmatrix}1\\1\\2\\-1\end{bmatrix}$$

In [345…
```python
L1, U1 = descomposicion_LU(np.array(A))
b1 = np.array([1,1,2,-1])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  3. -1. -2.]
 [ 0. -1.  1. -1.]]
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0. -2.  1.]]
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0.  0. -1.]]
[01-25 12:44:50][INFO]
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0.  0. -1.]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[ 1.]
 [ 0.]
 [ 0.]
 [-1.]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 2
[01-25 12:44:50][INFO] suma = [-8.]
[01-25 12:44:50][INFO] U[i, i] = 8.0
[01-25 12:44:50][INFO] y[i] = [0.]
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [-1.]
[01-25 12:44:50][INFO] U[i, i] = 1.0
[01-25 12:44:50][INFO] y[i] = [0.]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [0.]
[01-25 12:44:50][INFO] U[i, i] = 1.0
[01-25 12:44:50][INFO] y[i] = [1.]
Matriz L:
[[ 1.    0.    0.    0.  ]
 [ 1.    1.    0.    0.  ]
 [ 2.    3.    1.    0.  ]
 [ 0.   -1.   -0.25  1.  ]]
Matriz U:
[[ 1. -1.  2. -1.]
 [ 0.  1. -3.  2.]
 [ 0.  0.  8. -8.]
 [ 0.  0.  0. -1.]]
Solución del sistema Ax=b:
[[1.]
 [1.]
 [1.]
 [1.]]
```

4)

Encuentre los valores de A que hacen que la siguiente matriz sea singular.

$$\begin{bmatrix}1 & -1 & \alpha\\2 & 2 & 1\\0 & \alpha & -\frac{3}{2}\end{bmatrix}$$

In [346…
```python
import sympy as sp

alpha = sp.symbols('alpha')

A = sp.Matrix([
    [1, -1, alpha],
    [2,  2, 1],
    [0, alpha, -sp.Rational(3, 2)]
])

detA = A.det()

print("Determinante:")
sp.pprint(detA)

valores_alpha = sp.solve(detA, alpha)

print("\nValores de alpha que hacen singular la matriz:")
print(valores_alpha)
```
```
Determinante:
   2
2⋅α  - α - 6

Valores de alpha que hacen singular la matriz:
[-3/2, 2]
```

5)

Resuelva los siguientes sistemas lineales:

a.

$$\begin{bmatrix}1 & 0 & 0\\2 & 1 & 0\\-1 & 0 & 1\end{bmatrix}\begin{bmatrix}2 & 3 & -1\\0 & -2 & 1\\0 & 0 & 3\end{bmatrix}\begin{bmatrix}x_1\\x_2\\x_3\end{bmatrix} = \begin{bmatrix}2\\-1\\1\end{bmatrix}$$

In [347…
```python
A= np.array([[1, 0, 0],
             [2, 1, 0],
             [-1, 0, 1]])

B = np.array([[2, 3, -1],
             [0, -2, 1],
             [0, 0, 3]])

C = A @ B
```

In [348…
```python
L1, U1 = descomposicion_LU(np.array(C))
b1 = np.array([2,-1,1])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[ 2.  3. -1.]
 [ 0. -2.  1.]
 [ 0.  0.  3.]]
[01-25 12:44:50][INFO]
[[ 2.  3. -1.]
 [ 0. -2.  1.]
 [ 0.  0.  3.]]
[01-25 12:44:50][INFO]
[[ 2.  3. -1.]
 [ 0. -2.  1.]
 [ 0.  0.  3.]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[ 2.]
 [-5.]
 [ 3.]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [1.]
[01-25 12:44:50][INFO] U[i, i] = -2.0
[01-25 12:44:50][INFO] y[i] = [-5.]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [8.]
[01-25 12:44:50][INFO] U[i, i] = 2.0
[01-25 12:44:50][INFO] y[i] = [2.]
Matriz L:
[[ 1.  0.  0.]
 [ 2.  1.  0.]
 [-1. -0.  1.]]
Matriz U:
[[ 2.  3. -1.]
 [ 0. -2.  1.]
 [ 0.  0.  3.]]
Solución del sistema Ax=b:
[[-3.]
 [ 3.]
 [ 1.]]
```

b.

$$\begin{bmatrix}2 & 0 & 0\\-1 & 1 & 0\\3 & 2 & -1\end{bmatrix}\begin{bmatrix}1 & 1 & 1\\0 & 1 & 2\\0 & 0 & 1\end{bmatrix}\begin{bmatrix}x_1\\x_2\\x_3\end{bmatrix} = \begin{bmatrix}-1\\3\\0\end{bmatrix}$$

In [349…
```python
A= np.array([[2, 0, 0],
             [-1, 1, 0],
             [3, 2, -1]])

B = np.array([[1, 1, 1],
             [0, 1, 2],
             [0, 0, 1]])

C = A @ B
```

In [350…
```python
L1, U1 = descomposicion_LU(np.array(C))
b1 = np.array([-1,3,0])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[2. 2. 2.]
 [0. 1. 2.]
 [0. 2. 3.]]
[01-25 12:44:50][INFO]
[[ 2.  2.  2.]
 [ 0.  1.  2.]
 [ 0.  0. -1.]]
[01-25 12:44:50][INFO]
[[ 2.  2.  2.]
 [ 0.  1.  2.]
 [ 0.  0. -1.]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[-1. ]
 [ 2.5]
 [-3.5]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [7.]
[01-25 12:44:50][INFO] U[i, i] = 1.0
[01-25 12:44:50][INFO] y[i] = [2.5]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [-2.]
[01-25 12:44:50][INFO] U[i, i] = 2.0
[01-25 12:44:50][INFO] y[i] = [-1.]
Matriz L:
[[ 1.   0.   0. ]
 [-0.5  1.   0. ]
 [ 1.5  2.   1. ]]
Matriz U:
[[ 2.  2.  2.]
 [ 0.  1.  2.]
 [ 0.  0. -1.]]
Solución del sistema Ax=b:
[[ 0.5]
 [-4.5]
 [ 3.5]]
```

6)

Factorice las siguientes matrices en la descomposición LU mediante el algoritmo de factorización LU con lii = 1 para todas las i.

a.

$$\begin{bmatrix}2 & -1 & 1\\3 & 3 & 9\\3 & 3 & 5\end{bmatrix}$$

In [351…
```python
A = np.array([[2, -1, 1],
              [3, 3, 9],
                [3, 3, 5]])
```

In [352…
```python
L,U = descomposicion_LU(A)
print("Matriz L:")
print(L)
print("Matriz U:")
print(U)
```
```
[01-25 12:44:50][INFO]
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   4.5  3.5]]
[01-25 12:44:50][INFO]
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   0.  -4. ]]
[01-25 12:44:50][INFO]
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   0.  -4. ]]
Matriz L:
[[1.  0.  0. ]
 [1.5 1.  0. ]
 [1.5 1.  1. ]]
Matriz U:
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   0.  -4. ]]
```

b.

$$\begin{bmatrix}1.012 & -2.132 & 3.104\\-2.132 & 4.096 & -7.013\\3.104 & -7.013 & 0.014\end{bmatrix}$$

In [353…
```python
B = np.array([[1.012, -2.132, 3.104],
              [-2.132, 4.096, -7.013],
                [3.104, -7.013, 0.014]])
```

In [354…
```python
L,U = descomposicion_LU(B)
print("Matriz L:")
print(L)
print("Matriz U:")
print(U)
```
```
[01-25 12:44:50][INFO]
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.         -0.47374308 -9.50656917]]
[01-25 12:44:50][INFO]
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.          0.         -8.93914077]]
[01-25 12:44:50][INFO]
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.          0.         -8.93914077]]
Matriz L:
[[ 1.          0.          0.        ]
 [-2.10671937  1.          0.        ]
 [ 3.06719368  1.19775553  1.        ]]
Matriz U:
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.          0.         -8.93914077]]
```

c.

$$\begin{bmatrix}2 & 0 & 0 & 0\\1 & 1.5 & 0 & 0\\0 & -3 & 0.5 & 0\\2 & -2 & 1 & 1\end{bmatrix}$$

In [355…
```python
C = np.array([[2, 0, 0, 0],
             [1, 1.5, 0, 0],
                 [0, -3, 0.5, 0],
                 [2, -2, 1, 1]])
```

In [356…
```python
L,U = descomposicion_LU(C)
print("Matriz L:")
print(L)
print("Matriz U:")
print(U)
```
```
[01-25 12:44:50][INFO]
[[ 2.   0.   0.   0. ]
 [ 0.   1.5  0.   0. ]
 [ 0.  -3.   0.5  0. ]
 [ 0.  -2.   1.   1. ]]
[01-25 12:44:50][INFO]
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  1.  1. ]]
[01-25 12:44:50][INFO]
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  0.  1. ]]
[01-25 12:44:50][INFO]
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  0.  1. ]]
Matriz L:
[[ 1.          0.          0.          0.        ]
 [ 0.5         1.          0.          0.        ]
 [ 0.         -2.          1.          0.        ]
 [ 1.         -1.33333333  2.          1.        ]]
Matriz U:
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  0.  1. ]]
```

d.

$$\begin{bmatrix}2.1756 & 4.0231 & -2.1732 & 5.1967\\-4.0231 & 6.0000 & 0 & 1.1973\\-1.0000 & -5.2107 & 1.1111 & 0\\6.0235 & 7.0000 & 0 & -4.1561\end{bmatrix}$$

In [357…
```python
D = np.array([[2.1756, 4.0231, -2.1732, 5.1967],
              [-4.0231, 6.0000, 0, 1.973],
              [-1.0000, -5.2107, 1.1111, 0],
              [6.0235, 7.0000, 0, -4.1561]])
```

In [358…
```python
L,U = descomposicion_LU(D)
print("Matriz L:")
print(L)
print("Matriz U:")
print(U)
```
```
[01-25 12:44:50][INFO]
[[  2.1756       4.0231      -2.1732       5.1967    ]
 [  0.          13.43948042  -4.01866194  11.58269101]
 [  0.          -3.36150897   0.11220314   2.38862842]
 [  0.          -4.13860216   6.01685521 -18.54400331]]
[01-25 12:44:50][INFO]
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  4.77933394e+00 -1.49771876e+01]]
[01-25 12:44:50][INFO]
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.33134477e+01]]
[01-25 12:44:50][INFO]
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.33134477e+01]]
Matriz L:
[[ 1.          0.          0.          0.        ]
 [-1.84919103  1.          0.          0.        ]
 [-0.45964332 -0.25012194  1.          0.        ]
 [ 2.76866152 -0.30794361 -5.35228302  1.        ]]
Matriz U:
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.33134477e+01]]
```

7)

Modifique el algoritmo de eliminación gaussiana de tal forma que se pueda utilizar para resolver un sistema lineal usando la descomposición LU y, a continuación, resuelva los siguientes sistemas lineales.

$2x_1 - x_2 + x_3 = 1$
$3x_1 + 3x_2 + 9x_3 = 0$
$3x_1 + 3x_2 + 5x_3 = 4$

In [359…
```python
A = np.array([[2,-1,1],
              [3,3,9],
                [3,3,5]])
```

In [360…
```python
L1, U1 = descomposicion_LU(np.array(A))
b1 = np.array([1,0,4])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   4.5  3.5]]
[01-25 12:44:50][INFO]
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   0.  -4. ]]
[01-25 12:44:50][INFO]
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   0.  -4. ]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[ 1. ]
 [-1.5]
 [ 4. ]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [-7.5]
[01-25 12:44:50][INFO] U[i, i] = 4.5
[01-25 12:44:50][INFO] y[i] = [-1.5]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [-2.33333333]
[01-25 12:44:50][INFO] U[i, i] = 2.0
[01-25 12:44:50][INFO] y[i] = [1.]
Matriz L:
[[1.  0.  0. ]
 [1.5 1.  0. ]
 [1.5 1.  1. ]]
Matriz U:
[[ 2.  -1.   1. ]
 [ 0.   4.5  7.5]
 [ 0.   0.  -4. ]]
Solución del sistema Ax=b:
[[ 1.66666667]
 [ 1.33333333]
 [-1.        ]]
```

$1.012x_1 - 2.132x_2 + 3.104x_3 = 1.984$
$-2.132x_1 + 4.096x_2 - 7.013x_3 = -5.049$
$3.104x_1 - 7.013x_2 + 0.014x_3 = -3.895$

In [361…
```python
B = np.array([[1.012, -2.132, 3.104],
              [-2.132, 4.096, -7.013],
                [3.104, -7.013, 0.014]])
```

In [362…
```python
L1, U1 = descomposicion_LU(np.array(B))
b1 = np.array([1.984, -5.049, -3.895])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.         -0.47374308 -9.50656917]]
[01-25 12:44:50][INFO]
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.          0.         -8.93914077]]
[01-25 12:44:50][INFO]
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.          0.         -8.93914077]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[ 1.984     ]
 [-0.86926877]
 [-8.93914077]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [-0.47374308]
[01-25 12:44:50][INFO] U[i, i] = -0.3955256916996053
[01-25 12:44:50][INFO] y[i] = [-0.86926877]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [0.972]
[01-25 12:44:50][INFO] U[i, i] = 1.012
[01-25 12:44:50][INFO] y[i] = [1.984]
Matriz L:
[[ 1.          0.          0.        ]
 [-2.10671937  1.          0.        ]
 [ 3.06719368  1.19775553  1.        ]]
Matriz U:
[[ 1.012      -2.132       3.104     ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.          0.         -8.93914077]]
Solución del sistema Ax=b:
[[1.]
 [1.]
 [1.]]
```

$2x_1 = 3$
$x_1 + 1.5x_2 = 4.5$
$-3x_2 + 0.5x_3 = -6.6$
$2x_1 - 2x_2 + x_3 + x_4 = 0.8$

In [363…
```python
C = np.array ([[2, 0, 0, 0],
              [1, 1.5, 0, 0],
              [0, -3, 0.5, 0],
               [2, -2, 1, 1]])
```

In [364…
```python
L1, U1 = descomposicion_LU(np.array(C))
b1 = np.array([3, 4.5, -6.6, 0.8])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[ 2.   0.   0.   0. ]
 [ 0.   1.5  0.   0. ]
 [ 0.  -3.   0.5  0. ]
 [ 0.  -2.   1.   1. ]]
[01-25 12:44:50][INFO]
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  1.  1. ]]
[01-25 12:44:50][INFO]
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  0.  1. ]]
[01-25 12:44:50][INFO]
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  0.  1. ]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[ 3. ]
 [ 3. ]
 [-0.6]
 [ 3. ]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 2
[01-25 12:44:50][INFO] suma = [0.]
[01-25 12:44:50][INFO] U[i, i] = 0.5
[01-25 12:44:50][INFO] y[i] = [-0.6]
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [0.]
[01-25 12:44:50][INFO] U[i, i] = 1.5
[01-25 12:44:50][INFO] y[i] = [3.]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [0.]
[01-25 12:44:50][INFO] U[i, i] = 2.0
[01-25 12:44:50][INFO] y[i] = [3.]
Matriz L:
[[ 1.          0.          0.          0.        ]
 [ 0.5         1.          0.          0.        ]
 [ 0.         -2.          1.          0.        ]
 [ 1.         -1.33333333  2.          1.        ]]
Matriz U:
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  0.  1. ]]
Solución del sistema Ax=b:
[[ 1.5]
 [ 2. ]
 [-1.2]
 [ 3. ]]
```

$2.1756x_1 + 4.0231x_2 - 2.1732x_3 + 5.1967x_4 = 17.102$
$-4.0231x_1 + 6.0000x_2 + 1.1973x_4 = -6.1593$
$-1.0000x_1 - 5.2107x_2 + 1.1111x_3 = 3.0004$
$6.0235x_1 + 7.0000x_2 - 4.1561x_4 = 0.0000$

In [365…
```python
D = np.array([[2.1756, 4.0231, -2.1732, 5.1967],
              [
              -4.0231, 6.0000, 0, 1.973],
              [-1.0000, -5.2107, 1.1111, 0],
              [6.0235, 7.0000, 0, -4.1561]])
```

In [366…
```python
L1, U1 = descomposicion_LU(np.array(D))
b1 = np.array([17.102, -6.1593, 3.0004, 0.0000])
x1 = resolver_LU(L1, U1, b1)
print("Matriz L:")
print(L1)
print("Matriz U:")
print(U1)
print("Solución del sistema Ax=b:")
print(x1)
```
```
[01-25 12:44:50][INFO]
[[  2.1756       4.0231      -2.1732       5.1967    ]
 [  0.          13.43948042  -4.01866194  11.58269101]
 [  0.          -3.36150897   0.11220314   2.38862842]
 [  0.          -4.13860216   6.01685521 -18.54400331]]
[01-25 12:44:50][INFO]
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  4.77933394e+00 -1.49771876e+01]]
[01-25 12:44:50][INFO]
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.33134477e+01]]
[01-25 12:44:50][INFO]
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.33134477e+01]]
[01-25 12:44:50][INFO] Sustitución hacia adelante
[01-25 12:44:50][INFO] y =
[[17.102     ]
 [25.46556496]
 [17.23071662]
 [52.71598078]]
[01-25 12:44:50][INFO] Sustitución hacia atrás
[01-25 12:44:50][INFO] i = 2
[01-25 12:44:50][INFO] suma = [20.92933286]
[01-25 12:44:50][INFO] U[i, i] = -0.8929523938192969
[01-25 12:44:50][INFO] y[i] = [17.23071662]
[01-25 12:44:50][INFO] i = 1
[01-25 12:44:50][INFO] suma = [29.21753865]
[01-25 12:44:50][INFO] U[i, i] = 13.439480423791139
[01-25 12:44:50][INFO] y[i] = [25.46556496]
[01-25 12:44:50][INFO] i = 0
[01-25 12:44:50][INFO] suma = [10.45231075]
[01-25 12:44:50][INFO] U[i, i] = 2.1756
[01-25 12:44:50][INFO] y[i] = [17.102]
Matriz L:
[[ 1.          0.          0.          0.        ]
 [-1.84919103  1.          0.          0.        ]
 [-0.45964332 -0.25012194  1.          0.        ]
 [ 2.76866152 -0.30794361 -5.35228302  1.        ]]
Matriz U:
[[ 2.17560000e+00  4.02310000e+00 -2.17320000e+00  5.19670000e+00]
 [ 0.00000000e+00  1.34394804e+01 -4.01866194e+00  1.15826910e+01]
 [ 0.00000000e+00  4.44089210e-16 -8.92952394e-01  5.28571362e+00]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.33134477e+01]]
Solución del sistema Ax=b:
[[ 3.05648522]
[-0.2791755 ]
[ 4.14200831]
[ 3.95960401]]
```